

Sprint 9: Routing, React Frameworks, and Server Components

1. Current progress

You have already covered the core React foundation:

- Sprint 1: React orientation, SPAs, and Vite setup
- Sprint 2: JSX, components, fragments, exports, imports, and folder structure
- Sprint 3: props, composition, conditions, lists, keys, inline styles, and accessibility
- Sprint 4: events and form inputs
- Sprint 5: `useState`, rendering, immutable updates, and beginner `useReducer`
- Sprint 6: refs, effects, cleanup, custom hooks, and Context API
- Sprint 7: data fetching, async UI, backend integration basics, action state, and optimistic UI
- Sprint 8: memoization, `useMemo`, `useCallback`, `React.memo`, and dependency management

So far, most of your React work has been about building components, passing data, managing state, handling user actions, fetching data, and avoiding unnecessary work.

Sprint 9 is the point where your app becomes a multi-page-style application.

React is still running as one app, but different URLs show different views.

2. Sprint 9 goal

By the end of Sprint 9, you should understand:

- What routing is
- Why SPAs need routing
- How to install and use React Router
- How to create normal routes
- How to create dynamic routes
- How to read URL params with `useParams`
- How to create nested routes
- How to use `Outlet`
- How to create index routes
- How to create a 404 route
- How to create a protected route
- How to lazy-load route pages with `React.lazy` and `Suspense`
- Why frameworks like Next.js exist
- What Server Components are at a beginner level

Simple version:

Sprint 9 teaches you how to connect URLs to React components.

3. What problem does routing solve?

Before routing, your React app may feel like this:

```
One app  
One main visible screen  
Everything controlled inside App.jsx
```

After routing, your app can feel like this:

```
/           -> Home page  
/about      -> About page  
/products   -> Products page  
/products/12 -> Product detail page  
/dashboard  -> Dashboard page  
/random-url -> 404 page
```

Routing means:

When the URL changes, React shows the matching component.

Example:

```
URL: /about  
Show: <About />  
  
URL: /products  
Show: <Products />  
  
URL: /products/keyboard  
Show: <ProductDetail />
```

4. React Router installation

For a normal Vite React app, install React Router:

```
npm i react-router
```

Then you can use:

```
import { BrowserRouter, Routes, Route } from "react-router";
```

5. The three main routing pieces

BrowserRouter

`BrowserRouter` gives your app routing support using the browser URL.

Think of it as:

```
BrowserRouter = gives the app routing powers
```

Usually it wraps the routes.

Routes

`Routes` contains all the route rules.

Think of it as:

```
Routes = the list of possible routes
```

Route

`Route` connects a URL path to a component.

Think of it as:

```
Route = if this path matches, show this component
```

Basic example:

```
import { BrowserRouter, Routes, Route } from "react-router";
```

```
function Home() {
  return <h1>Home Page</h1>;
}

function About() {
  return <h1>About Page</h1>;
}

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Read this:

```
<Route path="/about" element={<About />} />
```

as:

When the URL is /about, show the About component.

6. Sprint 9 folder structure

Use route-level files in `pages/` and reusable UI in `components/`.

```
src/
  App.jsx
  pages/
    Home.jsx
    About.jsx
    Products.jsx
    ProductDetail.jsx
    CourseLayout.jsx
    CourseOverview.jsx
    CourseLessons.jsx
    Dashboard.jsx
```

```
Login.jsx
NotFound.jsx
components/
  Navbar.jsx
  ProtectedRoute.jsx
```

Beginner rule:

```
components/ = reusable UI pieces
pages/ = full route screens
```

7. Basic routes

Create these pages:

```
// src/pages/Home.jsx
export default function Home() {
  return <h1>Home</h1>;
}
```

```
// src/pages/About.jsx
export default function About() {
  return <h1>About</h1>;
}
```

```
// src/pages/Products.jsx
export default function Products() {
  return <h1>Products</h1>;
}
```

Use them in `App.jsx`:

```
import { BrowserRouter, Routes, Route } from "react-router";

import Home from "../pages/Home";
import About from "../pages/About";
import Products from "../pages/Products";

export default function App() {
  return (
```

```

    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/products" element={<Products />} />
      </Routes>
    </BrowserRouter>
  );
}

```

Now:

```

/           -> Home
/about      -> About
/products   -> Products

```

8. Dynamic routes

A dynamic route has a URL part that changes.

Example:

```

/products/keyboard
/products/mouse
/products/monitor

```

Instead of writing many routes, write one route:

```

<Route path="/products/:productId" element={<ProductDetail />} />

```

The `:productId` part is a variable URL segment.

Create this page:

```

// src/pages/ProductDetail.jsx
import { useParams } from "react-router";

export default function ProductDetail() {
  const { productId } = useParams();

```

```
    return (  
      <main>  
        <h1>Product Detail</h1>  
        <p>Product ID: {productId}</p>  
      </main>  
    );  
  }  
}
```

Add the route:

```
<Route path="/products/:productId" element={<ProductDetail />} />
```

Now:

```
/products/keyboard -> productId is "keyboard"  
/products/42       -> productId is "42"  
/products/mouse    -> productId is "mouse"
```

Important:

URL params are strings.

9. Nested routes

Nested routes are routes inside other routes.

Example:

```
/courses/react  
/courses/react/lessons
```

This means:

```
courses  
├── courseId  
│   ├── overview  
│   └── lessons
```

React Router uses `Outlet` to show the child route inside the parent route.

Create the parent layout:

```
// src/pages/CourseLayout.jsx
import { Outlet, useParams } from "react-router";

export default function CourseLayout() {
  const { courseId } = useParams();

  return (
    <main>
      <h1>Course: {courseId}</h1>
      <Outlet />
    </main>
  );
}
```

Create child pages:

```
// src/pages/CourseOverview.jsx
export default function CourseOverview() {
  return <p>This is the course overview.</p>;
}
```

```
// src/pages/CourseLessons.jsx
export default function CourseLessons() {
  return <p>These are the course lessons.</p>;
}
```

Add nested routes:

```
<Route path="/courses/:courseId" element={<CourseLayout />}>
  <Route index element={<CourseOverview />} />
  <Route path="lessons" element={<CourseLessons />} />
</Route>
```

Now:

```
/courses/react          -> CourseLayout + CourseOverview
/courses/react/lessons -> CourseLayout + CourseLessons
```

The `index` route means:

Show this child when the parent route matches exactly.

10. 404 route

A 404 page appears when no route matches.

Create:

```
// src/pages/NotFound.jsx
export default function NotFound() {
  return (
    <main>
      <h1>404</h1>
      <p>Page not found.</p>
    </main>
  );
}
```

Add this route:

```
<Route path="*" element={<NotFound />} />
```

The `*` catches unknown routes.

Examples:

```
/anything-random -> NotFound
/wrong/page      -> NotFound
```

11. Protected route

A protected route blocks private pages if the user is not logged in.

Example:

```
/dashboard
```

should only show if:

```
isLoggedIn === true
```

Create:

```
// src/components/ProtectedRoute.jsx
import { Navigate } from "react-router";

export default function ProtectedRoute({ isLoggedIn, children }) {
  if (!isLoggedIn) {
    return <Navigate to="/login" replace />;
  }

  return children;
}
```

Use it:

```
<Route
  path="/dashboard"
  element={
    <ProtectedRoute isLoggedIn={isLoggedIn}>
      <Dashboard />
    </ProtectedRoute>
  }
/>
```

Mental model:

```
ProtectedRoute is a gate.
If logged in: let the user enter.
If not logged in: send the user to login.
```

12. Lazy loading with `React.lazy()` and `Suspense`

Normally, imported components are loaded immediately.

Lazy loading means:

Do not load this page until the user needs it.

This is useful for large pages.

Example:

```
import { lazy, Suspense } from "react";

const Dashboard = lazy(() => import("./pages/Dashboard"));
```

Use it:

```
<Suspense fallback={<p>Loading page...</p>}>
  <Dashboard />
</Suspense>
```

Inside a route:

```
<Route
  path="/dashboard"
  element={
    <ProtectedRoute isLoggedIn={isLoggedIn}>
      <Suspense fallback={<p>Loading dashboard...</p>}>
        <Dashboard />
      </Suspense>
    </ProtectedRoute>
  }
/>
```

Beginner warning:

For now, use Suspense for lazy-loading components. Do not jump into advanced Suspense data fetching yet.

13. Complete Sprint 9 example

```
// src/App.jsx
import { BrowserRouter, Routes, Route } from "react-router";
import { lazy, Suspense } from "react";
```

```

import Home from "../pages/Home";
import About from "../pages/About";
import Products from "../pages/Products";
import ProductDetail from "../pages/ProductDetail";
import Login from "../pages/Login";
import NotFound from "../pages/NotFound";
import CourseLayout from "../pages/CourseLayout";
import CourseOverview from "../pages/CourseOverview";
import CourseLessons from "../pages/CourseLessons";
import ProtectedRoute from "../components/ProtectedRoute";

const Dashboard = lazy(() => import("../pages/Dashboard"));

export default function App() {
  const isLoggedIn = false;

  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />

        <Route path="/products" element={<Products />} />
        <Route path="/products/:productId" element={<ProductDetail />} />

        <Route path="/courses/:courseId" element={<CourseLayout />}>
          <Route index element={<CourseOverview />} />
          <Route path="lessons" element={<CourseLessons />} />
        </Route>

        <Route path="/login" element={<Login />} />

        <Route
          path="/dashboard"
          element={
            <ProtectedRoute isLoggedIn={isLoggedIn}>
              <Suspense fallback={<p>Loading dashboard...</p>}>
                <Dashboard />
              </Suspense>
            </ProtectedRoute>
          }
        />

        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

```

This example includes:

- Normal routes
- Dynamic routes
- Nested routes
- Index route
- 404 route
- Protected route
- Lazy-loaded route

14. React frameworks and Next.js readiness

React is a UI library.

A framework like Next.js adds more structure and app-level features.

Beginner comparison:

Tool	Main job
React	Build UI components
React Router	Add client-side routes to a React app
Next.js	Add routing, server rendering, data patterns, backend-like route handlers, image optimization, and stronger app structure

Do not jump fully into Next.js yet.

Sprint 9 only wants you to understand why frameworks exist and what problems they solve.

15. Server Components, beginner version

A normal client component runs in the browser.

It can use:

```
useState
useEffect
onClick
localStorage
```

window
browser APIs

A Server Component renders on the server.

Beginner mental model:

Server Component:
Good for reading/fetching data and returning UI.

Client Component:
Good for buttons, forms, state, effects, and browser interaction.

In Next.js-style projects, interactive components often need this at the top:

```
"use client";
```

Do not go deep yet. Just understand the separation:

Server = data and rendering work before the browser
Client = browser interaction and live UI behavior

16. Debug task

Broken code:

```
import { Routes, Route, useParams } from "react-router";

function Product() {
  const { id } = useParams();
  return <h1>{id}</h1>;
}

export default function App() {
  return <Routes><Route path="product/:id" component={<Product />} /></Routes>;
}
```

Problems:

useParam is wrong. It should be useParams.
Routes must be inside BrowserRouter.
Route uses element, not component.

Correct version:

```
import { BrowserRouter, Routes, Route, useParams } from "react-router";

function Product() {
  const { id } = useParams();

  return <h1>{id}</h1>;
}

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/product/:id" element={<Product />} />
      </Routes>
    </BrowserRouter>
  );
}
```

17. Sprint 9 practice assignment

Build a Course Browser.

Required routes:

/	-> Home
/about	-> About
/courses	-> Courses list
/courses/:courseId	-> Course overview
/courses/:courseId/lessons	-> Course lessons
/dashboard	-> Protected dashboard
/login	-> Login
/random-url	-> 404

Required concepts:

BrowserRouter
Routes
Route
useParams
nested routes
Outlet
index route
Navigate
ProtectedRoute
React.lazy
Suspense
path="*"

Definition of done:

I can type a URL directly and the correct page opens.
I can read a route param with useParams.
I can explain how nested routes build URLs.
Unknown URLs show a 404 page.
Protected dashboard redirects to login when logged out.
One page is lazy-loaded with Suspense fallback.

18. Shortest mental model

Sprint 9 teaches routing.

A route connects a URL to a component.
BrowserRouter gives the app routing support.
Routes holds route definitions.
Route maps a path to UI.
useParams reads dynamic URL values.
Nested routes build child URLs under parent URLs.
Outlet shows child route UI inside parent route UI.
Protected routes block private screens.
A 404 route catches unknown URLs.
React.lazy and Suspense load large pages only when needed.
Frameworks like Next.js add more structure on top of React.